ENGINEERING REFERENCE · INTERNAL

unidb — Design & Architecture

`FORMAT_VERSION 11 · 19 WAL record types · 51 crash points · four data models · items 71–99 incorporated · generated 2026-07-20`

A single embedded storage & transaction engine in Rust that unifies four data models — relational rows, vector search, graph edges, and a durable event queue — over **one page store, one WAL, one buffer pool, one transaction manager**. One transaction touches all four atomically, because there is one node and one log.

> **The competitive thesis.** unidb's moat is eliminating the multi-system dual-write tax: "save row + embedding + graph edge + event" is *one* WAL append chain and *one* group-committed `fsync`, versus 3–4 network round-trips with no shared transaction across Postgres + a vector store + a graph DB + Kafka. The claim is workload-specific and benchmarked against the *replaced stack*, not one incumbent on its home turf.

Contents

1. System architecture

The embedded crate is primary; the REST server is an optional, feature-gated layer. The default build has **zero async dependencies** — the engine core is fully synchronous. Every layer sits on the storage triad (page store · WAL · buffer pool); the query and Cypher engines both read and write all four data models through one MVCC transaction path.

Multi-Model Embedded Database — System Architecture

Relational · Vector · Graph · Queue — one atomic transaction, one file on disk

Figure 1 — The layer stack. Arrows trace the primary data path; every write follows write-ahead ordering (D5): the WAL record is durable before the page is flushed.

1.1 The recurring architectural move — durability reuse

Vector postings, graph adjacency, full-text postings, LOB directories, and the heap's free-space map are **all the same** `DiskBTree`; edges, events, consumer offsets, and LOB chunks are **all ordinary rows** in synthetic `__…__` system tables. Each new model therefore inherits WAL logging, crash recovery, MVCC, vacuum, and replication for free — **one crash-recovery code path covers five engines.**

Engine	Core data structure	Durability mechanism
Storage / heap	Slotted 8 KiB pages, CLOCK frame table, DiskBTree FSM, HOT chains, InsertAccum	WAL-before-page (D5) + FPI · CRC at pool boundary
WAL & recovery	16 MiB segment files, 41-byte record header, 19 record types	fsync (group-committed), CRC32 everywhere, background sealer
Transactions	`Snapshot{xmin,xmax,active}`, wait-for graph, bulk lock elision	undo log + `WAL_TXN_*` records
SQL	Logical/physical plan trees, 1,024-entry plan cache, O(1) COUNT(*)	reads: n/a; statements are mini-txns
B-tree / full-text	Right-linked B-tree over standard pages, sort-then-bulk-load, batch DML maintenance	redo-only `WAL_INDEX` + `WAL_INDEX_INSERT`
Vector	DiskHnswIndex on standard pages + L0/vec caches + NodeCache gate + SIMD	`WAL_INDEX` per node page
Graph	`__edges__` heap table + adjacency DiskBTree	ordinary row WAL + `WAL_INDEX`
Event queue	`__events__` + `__consumers__` heap tables	ordinary row WAL, txn-atomic

2. One commit, four models

The headline flow. A single transaction inserts a row, maintains its index, assigns a vector posting, and captures an event — then commits **once**. All statement mini-transactions append to the WAL without individual fsyncs; the single `sync_up_to` at user-commit is the one durable point, and concurrent committers coalesce behind one `fsync` via leader election.

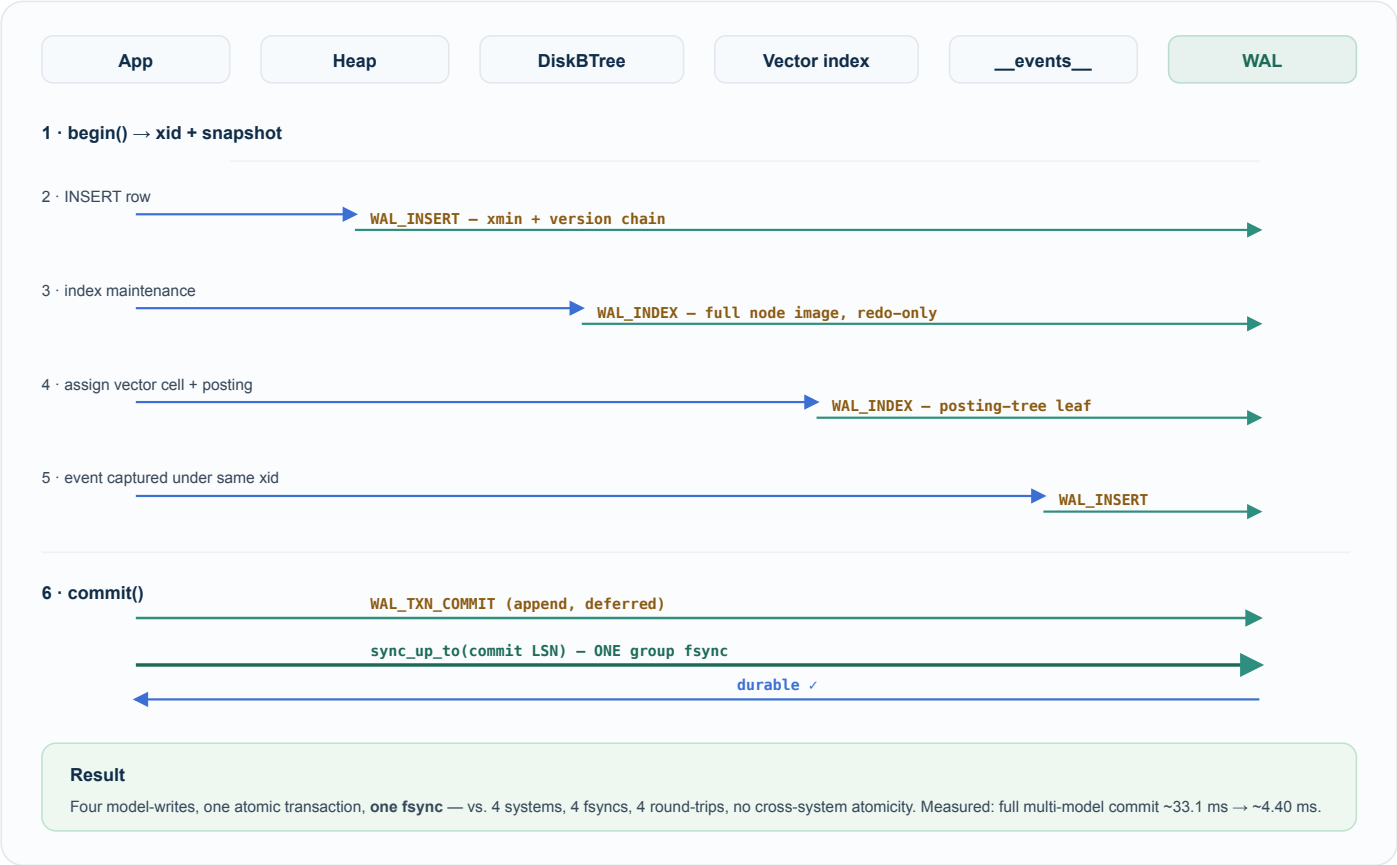

Figure 2 — The write path. Blue = call; green = WAL append; heavy green = the single group-committed fsync that makes the whole transaction durable.

3. Storage engine & the write path

mmap-as-storage. The single `data.db` is memory-mapped; it is the only module permitted `unsafe`. `write_page` mutates bytes directly in the mmap under a write lock; readers take an **owned copy** under the read lock. There is no separate frame buffer and therefore no "pool newer than disk" divergence — which is exactly what later made parallel scan trivial to make correct. The OS page cache is the read cache; no user-space cache duplicates it.

3.1 Page & tuple layout

Structure	Size	Fields
Page header	28 B	`page_id`, `page_type`, **CRC32** (whole page, this field zeroed), **LSN** (last WAL record applied), `slot_count`, `free_start`, `free_end`. Slot array grows up; tuple data grows down.
Tuple header (D4)	24 B	`xmin`, `xmax` (0 = live), `prev_page`, `prev_slot` — MVCC visibility + backward version chain; no separate undo segment.

CRC detects torn writes; the LSN gates redo idempotence and the D5 invariant. Fresh all-zero pages legitimately skip CRC.

3.2 Durable free-space map — the O(1)-open moat

The per-table page directory + free-space map is a `DiskBTree` keyed `page_id → free_bytes`. `Heap::open` is **O(1)** (a handle is just a meta-page id + page size); the insert path never loads the full directory — it probes cached pages then the durable append tail via one `max_entry` descent. The in-memory free-map is a hint that **never over-reports**; `acquire_page_for_insert` re-checks under the page latch, so a stale hint costs a retry, never a corruption. This removed both the old ~145k-row catalog-blob ceiling and the O(pages) per-allocation rewrite (insert cost went flat at ~17–28 µs/row).

3.3 HOT update chains & batched writes

Insert-new-version MVCC normally requires a B-tree update on every UPDATE. For non-indexed-column updates ("HOT-eligible"), the B-tree is skipped and the new version is chained from the old slot: **same-page HOT** puts the new version on the same page with a `hot_next` forward pointer; **cross-page HOT** (page full) writes the new version elsewhere and marks the old slot with the `HOT_NEXT_XPAGE = 0xFFFE` sentinel, leaving the B-tree pointing at the old slot which resolves the live version.

> **Phase ordering is a crash-safety property.** Batch HOT update runs **A→B→C**: (A) stamp `xmax` on old versions, (B) insert new versions on fill pages, (C) write forward chain pointers. The original B→A order was a shipped bug: Phase A failing after Phase B committed left orphaned new versions with no undo record. A→B→C makes every phase boundary recover correctly.

Fill-page cursor tracks the current open page across rows in a statement, replacing a per-row O(log n) FSM probe. **InsertAccum** streams a multi-row `INSERT … VALUES` into one `WAL_BEGIN` / `WAL_COMMIT` pair per heap page instead of per row; UNIQUE enforcement still runs per-row before each row is written.

3.4 Checkpoint (strictly ordered; control mutex never held across an fsync)

`wal.sync()` → `flush_all` → clear FPI tracking → log checkpoint (fsynced) → control-file update (`next_xid` captured *before* truncation) → truncate below `min(checkpoint_lsn, replication retain floor)`. Auto-checkpoint fires on time (60 s) or WAL size (64 MiB) but **only at a quiescent point** (`active_count() == 0`) so truncation can never discard an in-flight transaction's undo records.

4. WAL & crash recovery

Steal + no-force buffering (D1) requires **both redo and undo** logging. The WAL is a directory of fixed 16 MiB segments; a record is `[len][41-byte header + redo + undo + CRC32]` and is never split across segments. 19 record types are defined; each new type triggered a `FORMAT_VERSION` bump so older binaries hard-fail with `BadVersion` rather than silently misrecovering.

4.1 Group commit

One mutex covers LSN allocation + physical append, so concurrent appenders never interleave partial records. `sync_up_to(target)` fast-paths if already durable; otherwise the leader runs the slow `sync_all` **with the append lock released**, so other committers keep appending and one fsync makes every commit before it durable — which is why write throughput scales with writers (8 writers → 3.55–3.82×) instead of fsyncs. The **background sealer** moves the 16-MiB segment-rotation fsync off the append mutex entirely.

> **fsync failure is session-fatal (fsyncgate).** Any `fsync` / `msync` failure — real or injected — poisons the WAL/buffer pool permanently; `durable_lsn` never advances and every later durability call returns `DurabilityFailure`. The remedy is restart + recovery, never an in-session retry that could falsely succeed.

4.2 Recovery flow

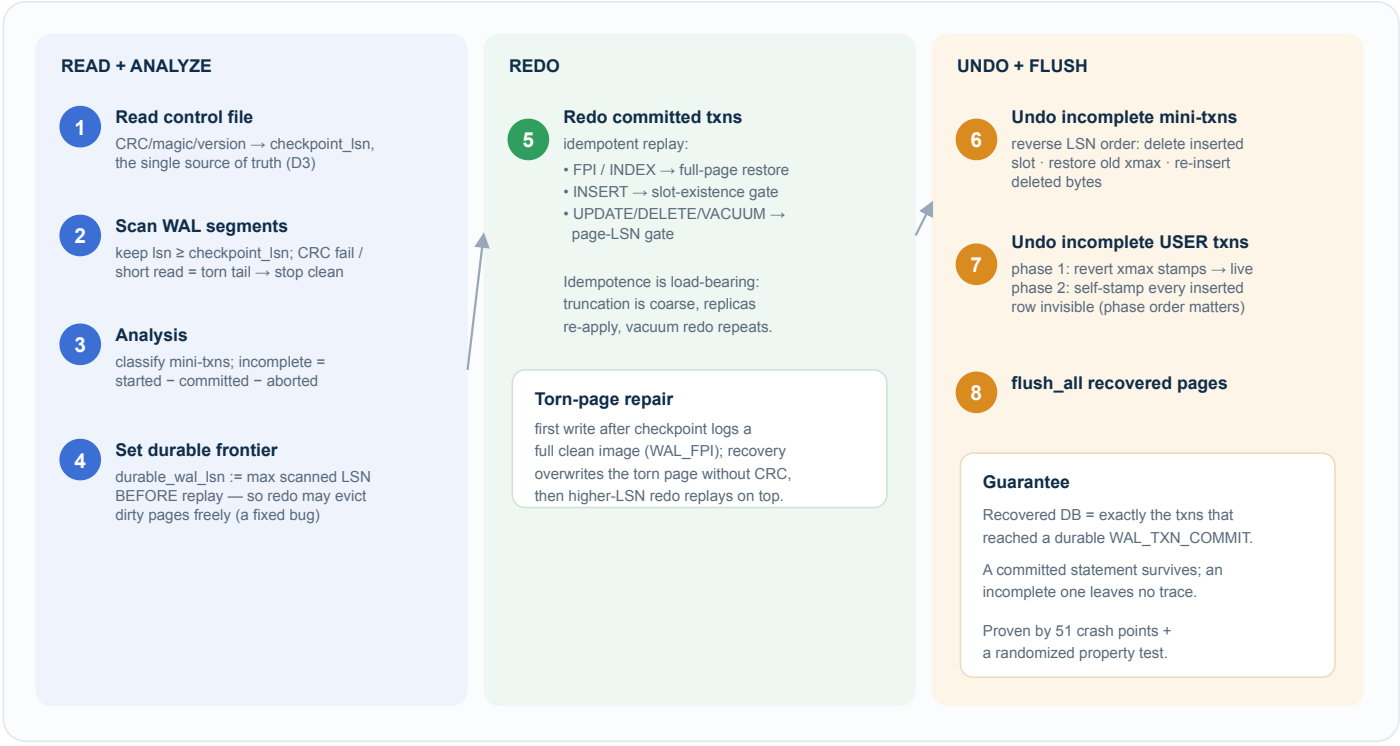

Figure 3 — Recovery starts at the control file (D3) and runs read → analyze → redo → undo → flush. Every redo record is idempotent; ownership in user-txn undo is derived from the tuple bytes (xmin/xmax), not a WAL field.

4.3 Crash-injection matrix (D7)

51 named kill points as of the current build, plus a randomized property test run under both durability policies. Crash coverage is a delivery criterion, not an afterthought — every storage-touching change adds points. Representative invariants:

Point	Kill site	Invariant proven
P1 / P5	WAL fsynced, page not flushed	redo restores the committed row
P11	genuine torn page on disk	FPI base + incremental redo restores both rows
P12	fsync fault (WAL commit + data msync)	refuses success; poisons; keeps failing
P13	`data.db` wiped entirely	B-tree rebuilt from `WAL_INDEX` images alone
P74 / P_xhot	crash mid batch / cross-page HOT update	committed batch survives; incomplete txn leaves originals visible
prop	random ops + random crash points, both policies	recovered DB = exactly the durably-committed txns

5. Transactions, MVCC & isolation

A snapshot is `{xmin, xmax, frozen active set}`. Read Committed recomputes it per statement; Repeatable Read and Serializable clone the BEGIN-time snapshot — the entire difference between isolation levels, over one visibility code path.

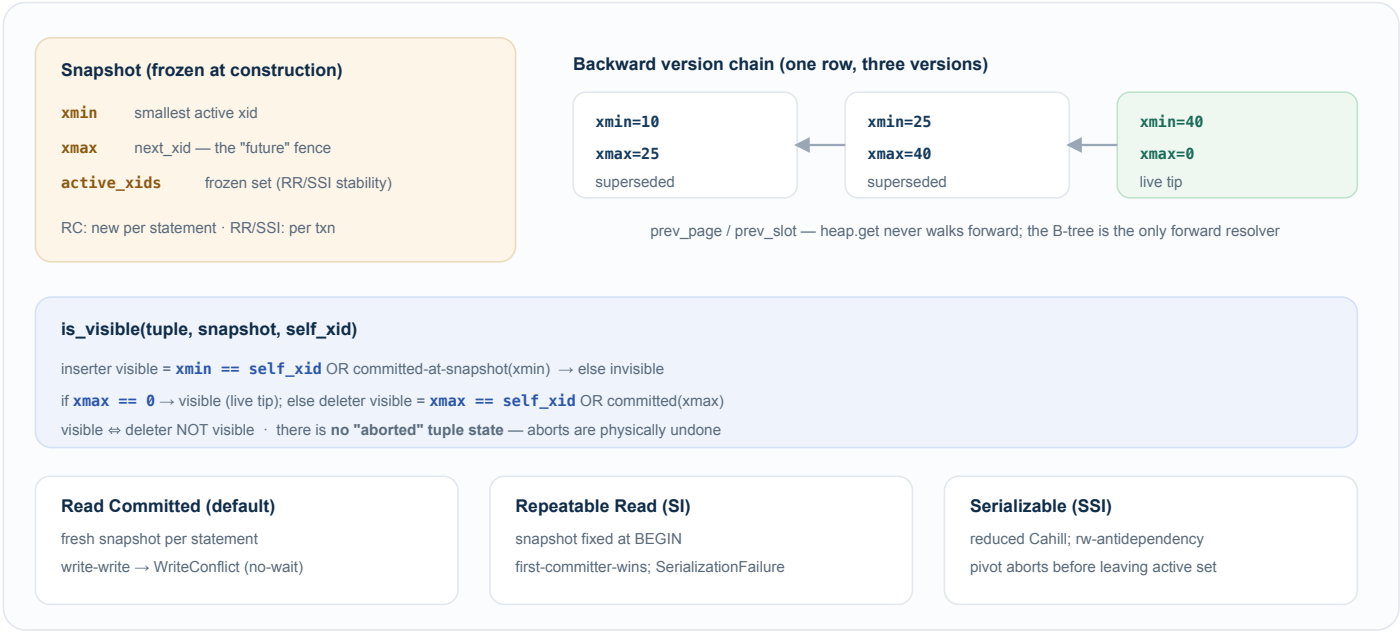

Figure 4 — MVCC visibility. A non-zero on-disk `xmax` always denotes a committed deleter (aborts are physically undone), which is what makes both visibility and vacuum reclaimability sound.

5.1 Bulk lock elision (item 88)

For batch DML, stamping `xmax = xid` under the page latch **is** the ownership claim — a concurrent writer sees `xmax != 0` and raises `WriteConflict`, so a separate lock-table entry is redundant. Elision drops lock-table mutex acquisitions from O(rows) to O(1) per batch (~50,000 round-trips saved at 50k rows). Undo still runs per-row. The gate: valid only under a single xid with every `xmax == 0` check already done atomically under the latch before stamping.

5.2 Vacuum & the index-aliasing gate

Vacuum gathers index keys *first* (tuple bytes are only readable while live), marks versions DEAD, then **scrubs every reclaimed RowId from all secondary indexes before** compacting DEAD → UNUSED. This gate is the single most important correctness step: a stale `(page, slot)` entry is harmless only while slots are never reused; the moment a slot is reused, a stale entry could resolve to a live, MVCC-visible, semantically wrong row. `WAL_VACUUM` is redo-only and idempotent, so a crash anywhere inside vacuum recovers cleanly.

6. Query, indexing, vector, graph & events

6.1 SQL query engine

Parse → `LogicalPlan` → optimizer → executors. A **1,024-entry plan cache** keyed by `(sql_hash, schema_epoch)` parses each unique statement once (schema DDL bumps the epoch to invalidate). **O(1) COUNT(*)** reads a maintained catalog `row_count` (bypassed for RLS-protected tables, which must count visible rows). GROUP BY pushes partial aggregates into parallel scan workers. Statements are mini-txns; `POST /batch-sql` runs up to 256 statements per round-trip.

6.2 Indexing — one DiskBTree, many roles

A right-linked B-tree over standard 8 KiB pages backs SQL indexes, full-text postings, vector postings, edge adjacency, and the heap FSM. Reads are latch-free; a stable meta-page id means no rebuild on open. Writes are redo-only full-node `WAL_INDEX` images, with a logical `WAL_INDEX_INSERT` for non-splitting leaves; `CREATE INDEX` uses sort-then-bulk-load (one mini-txn), and batch DML coalesces leaf writes.

6.3 Vector engine (HNSW)

The production index is an on-disk HNSW graph on standard pages. Query cost is dominated by per-hop node access, not the transaction wrapper — so the levers are caching and compute: an **L0 neighbour-list cache** and a **vector hot cache** (both prefetched after `CREATE INDEX`), zero-copy distance on cache hits, and an 8-lane SIMD-friendly distance accumulator. Warm NEAR latency at 10k×dim128 improved ~11× to ~2.38 ms; every index read still re-validates candidates against the caller's MVCC snapshot.

6.4 Graph engine

Edges are `(from_id, to_id, edge_type, props)` rows in the `__edges__` system table with an adjacency `DiskBTree` by `from_id`; a Cypher subset queries them. Full MVCC/SQL comes for free; the index is a hint that MVCC re-validates. Per-edge locking is automatic because lock keys are globally unique across the shared page pool.

6.5 Event queue

Events are rows in `__events__` captured **synchronously under the writer's xid**, so an event's fate is tied to its transaction — zero new abort-path code. Consumers track Kafka-style durable offsets; SSE subscriptions deliver sub-millisecond to idle subscribers. Because the queue is WAL-durable and executor-captured, the slim physical DML records (e.g. batch xmax) never need to reconstruct row content for a consumer.

7. Performance snapshot

Numbers come from shipped benchmark runs (100k rows, matched durability, real fsync on Linux); they are recorded, never invented. CRUD is benchmarked against Postgres for context — the headline claim is the multi-model one-commit path, not single-table CRUD.

Operation	unidb	vs Postgres	Note
DELETE all (truncate fast path)	27.2M rec/s	**8.0× faster**	O(pages) truncate vs per-row stamp
SELECT COUNT(*)	317M rec/s	**6.8× faster**	maintained catalog row_count
SELECT GROUP BY	24.7M rec/s	**1.4× faster**	parallel partial aggregates
DELETE selected	2.52M rec/s	0.81× (19% behind)	within target band
UPDATE (HOT-eligible)	453k rec/s	0.62× (38% behind)	CRC-boundary + fill-page cursor path
UPDATE (indexed column)	346k rec/s	0.42×	B-tree maintenance bound
Multi-model commit (4 writes)	~4.40 ms	—	one fsync vs a 4-system stack's four

> **Where the architecture wins outright:** large scans, counts, aggregates, and bulk truncation (1.4–8×), plus the structural multi-model advantage no single-engine tuning can replicate — one WAL, one commit, cross-model atomicity.

8. Trust & failure model

- **Every page read is CRC-checked** (fresh all-zero pages excepted); every WAL record is CRC-checked; the control file is CRC-checked and is the single source of recovery truth (D3).
- **D5 — WAL-before-page** is *the* invariant: no dirty page may be flushed or evicted while `page.LSN > durable_WAL_LSN`. Enforced at the eviction gate, re-checked at flush, guarded by a debug tripwire.
- **fsync failure is fatal for the session** — poisons the WAL/pool permanently; recovery via restart, never in-session retry.
- **Secondary indexes are hints, not truth.** Every index read re-validates candidates against the caller's MVCC snapshot; stale entries are harmless, missing entries are prevented by WAL-ordering, slot reuse is gated by vacuum's index scrub.
- **Scope discipline:** single-primary only (no distributed consensus), a practical SQL subset (not full ANSI), no cloud control plane. Every generalization costs throughput a specialized engine wouldn't pay.

9. Glossary

Term	Meaning
ARIES	Algorithm for Recovery and Isolation Exploiting Semantics — the redo/undo, WAL, steal + no-force recovery model unidb follows.
CRC32	Cyclic redundancy check on every page and WAL record; detects torn writes and corruption.

D1…D13	Locked design decisions (see `CLAUDE.md §3`). D5 = WAL-before-page; D3 = control file as recovery root.
FPI	Full-Page Image — a whole clean page logged on its first write after a checkpoint, for torn-page repair.
FSM	Free-Space Map — the durable `page_id → free_bytes` DiskBTree that gives O(1) heap open.
HNSW	Hierarchical Navigable Small World — the graph index behind vector similarity search.
HOT	Heap-Only Tuple — an UPDATE that chains a new version without touching the B-tree (no indexed column changed).
LSN	Log Sequence Number — monotonic WAL position; gates redo idempotence and the D5 frontier.
MVCC	Multi-Version Concurrency Control — readers see a consistent snapshot without blocking writers.
RC / RR / SSI	Read Committed / Repeatable Read / Serializable Snapshot Isolation — the three isolation levels.
SSE	Server-Sent Events — the push transport for event-queue subscriptions.
WAL	Write-Ahead Log — the durable redo+undo record stream; the source of crash recovery and replication.
xmin / xmax	Tuple header fields: the inserting and deleting/superseding transaction ids; `xmax = 0` means live.

---

unidb — Design & Architecture (engineering reference). Distilled from the `docs/design/processing-engines/` collection; when this document and `CLAUDE.md` / `PROGRESS.md` disagree, those win. Regeneration notes: `docs/design/unidb_design_architecture_context.md`.